


```

1 ### Small Creation 1 + Interval Reachability Tube
2 ### Earth → L1 Minimum-Energy Transfer (3D CR3BP) with Interval Tube Width Growth
3 ### Run in a Pluto notebook or as a plain Julia script
4
5 # =====
6 # Cell 1: Imports
7 # =====
8 begin
9     using Pkg
10    Pkg.activate(".")
11    Pkg.instantiate()
12    using InfiniteOpt, Ipopt, Plots, NLSolve, LinearAlgebra
13    using IntervalArithmetic      # ← added for interval reachability
14    # optional: using PlotlyJS; plotlyjs()
15 end
16
17 # =====
18 # Cell 2: Parameters (Earth-Moon)
19 # =====

```

```

Activating project at `~/julia/pluto_notebooks`
Precompiling project...
  x OrdinaryDiffEqDefault
  x OrdinaryDiffEq
  x TaylorIntegration → TaylorIntegrationDiffEqExt
0 dependencies successfully precompiled in 20 seconds. 468 already precompiled.

The following 2 direct dependencies failed to precompile:

TaylorIntegrationDiffEqExt

Failed to precompile TaylorIntegrationDiffEqExt [21fdf329-9f0e-520d-ae26-dab38dab1a79] to "/Users/nitishtyagi20/.julia/compiled/v1.11/TaylorIntegrationDiffEqExt/jl_Ae4k4w".
ERROR: LoadError: UndefVarError: `supports_solve_rng` not defined in `SciMLBase`
Stacktrace:
 [1] getproperty(x::Module, f::Symbol)
   @ Base ./Base.jl:42
 [2] top-level scope
   @ ~/.julia/packages/OrdinaryDiffEqDefault/yrn4p/src/default_alg.jl:50
 [3] include(mod::Module, _path::String)
   @ Base ./Base.jl:562
 [4] include(x::String)

```

```

1 begin
2    $\mu$  = 0.012150585609624          # Earth-Moon mass parameter
3   const u_max      = 0.1
4   const N_supports = 501
5   tf_lower  = 1.0
6   tf_upper  = 30.0
7   tf_start  = 15.0
8   println("μ = ",  $\mu$ , ", u_max = ", u_max, ", supports = ", N_supports)
9 end
10
11 # =====
12 # Cell 3: Compute L1 position
13 # =====

```

```
μ = 0.012150585609624, u_max = 0.1, supports = 501
```

```

1 begin
2   function dOmega_dx(x,  $\mu$ )
3     r1 = abs(x +  $\mu$ )
4     r2 = abs(x - (1 -  $\mu$ ))
5     return x - (1- $\mu$ )*(x +  $\mu$ )/r1^3 -  $\mu$ *(x - (1- $\mu$ ))/r2^3
6   end
7
8   res = nlsolve(x -> dOmega_dx(x[1],  $\mu$ ), [0.84]; ftol=1e-14)
9   if !NLSolve.converged(res)
10     @warn "L1 solver did not converge!"
11   end
12   x_L1 = res.zero[1]
13   println("Computed L1 x = ", x_L1)
14 end
15
16 # =====
17 # Cell 4: Model + variables - FULL 3D dynamics, uz = 0 exactly
18 # =====

```

```
Computed L1 x = 0.8369151257723574
```

An InfiniteOpt Model
 Feasibility problem with:
 Finite parameters: 0
 Infinite parameter: 1
 Variables: 10
 Derivatives: 0
 Measures: 0
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.LessThan{Float64}`: 3 constraints
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.EqualTo{Float64}`: 1 constraint
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.GreaterThan{Float64}`: 3 constraints
 Names registered in the model: tf, ux, uy, uz, vx, vy, vz, x, y, z, τ , $\partial\Omega_x$, $\partial\Omega_y$, $\partial\Omega_z$
 Transformation backend information:
 Backend type: TranscriptionBackend
 Solver: Ipopt
 Transformation built and up-to-date: false

```

1 begin
2     model = InfiniteModel(Ipopt.Optimizer)
3     @infinite_parameter(model,  $\tau \in [0, 1]$ , num_supports = N_supports)
4     @variable(model, tf_lower <= tf <= tf_upper, start = tf_start)
5
6     # Full 3D state variables
7     @variable(model, x, Infinite( $\tau$ ), start =  $\tau \rightarrow (-\mu + 0.01)*(1 - \tau) + x_{L1}*\tau$ )
8     @variable(model, y, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
9     @variable(model, z, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
10    @variable(model, vx, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
11    @variable(model, vy, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
12    @variable(model, vz, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
13
14    # Control: only in-plane thrust allowed
15    @variable(model, -u_max <= ux <= u_max, Infinite( $\tau$ ))
16    @variable(model, -u_max <= uy <= u_max, Infinite( $\tau$ ))
17    @variable(model, uz, Infinite( $\tau$ ))
18    fix.(uz, 0.0; force = true) # ← THE KEY: no out-of-plane control!
19
20    # Full 3D potential gradients
21    r1(x_, y_, z_) = sqrt((x_ +  $\mu$ )^2 + y_^2 + z_^2)
22    r2(x_, y_, z_) = sqrt((x_ - (1- $\mu$ ))^2 + y_^2 + z_^2)
23     $\partial\Omega_x(x_, y_, z_) = x_ - (1-\mu)*(x_ + \mu)/r1(x_, y_, z_)^3 - \mu*(x_ - (1-\mu))/r2(x_, y_, z_)^3$ 
24     $\partial\Omega_y(x_, y_, z_) = y_ * (1 - (1-\mu)/r1(x_, y_, z_)^3 - \mu/r2(x_, y_, z_)^3)$ 
25     $\partial\Omega_z(x_, y_, z_) = z_ * (1 - (1-\mu)/r1(x_, y_, z_)^3 - \mu/r2(x_, y_, z_)^3)$ 
26    model[: $\partial\Omega_x$ ] =  $\partial\Omega_x$ 
27    model[: $\partial\Omega_y$ ] =  $\partial\Omega_y$ 

```

```
28     model[: $\partial\Omega z$ ] =  $\partial\Omega z$ 
29     model
30 end
31
32 # =====
33 # Cell 5: Dynamics + boundary conditions + minimum-ENERGY objective
34 # =====
```

$$tf \times \int_{\tau \in [0,1]} ux(\tau)^2 + uy(\tau)^2 d\tau$$

```

1 begin
2   # Equations of motion (full 3D CR3BP)
3   @constraint(model, deriv(x, τ) == tf * vx)
4   @constraint(model, deriv(y, τ) == tf * vy)
5   @constraint(model, deriv(z, τ) == tf * vz)
6   @constraint(model, deriv(vx, τ) == tf * (2*vy + model[:∂Ωx](x,y,z) + ux))
7   @constraint(model, deriv(vy, τ) == tf * (-2*vx + model[:∂Ωy](x,y,z) + uy))
8   @constraint(model, deriv(vz, τ) == tf * (model[:∂Ωz](x,y,z) + uz)) # uz = 0
9
10  # Boundary conditions
11  @constraint(model, x(0) == -μ + 0.01)
12  @constraint(model, y(0) == 0)
13  @constraint(model, z(0) == 0)
14  @constraint(model, vx(0) == 0)
15  @constraint(model, vy(0) == 0)
16  @constraint(model, vz(0) == 0)
17
18  @constraint(model, x(1) == x_L1)
19  @constraint(model, y(1) == 0)
20  @constraint(model, z(1) == 0)
21  @constraint(model, vx(1) == 0)
22  @constraint(model, vy(1) == 0)
23  @constraint(model, vz(1) == 0)
24
25  # Minimum ENERGY objective (same as Small Creation 1)
26  @objective(model, Min, tf * ∫(ux^2 + uy^2, τ))
27 end
28
29 # =====
30 # Cell 6: Solve
31 # =====

```

```
1 begin
2   set_silent(model)    # optional: hide Ipopt log
3   @time optimize!(model)
4   println("Status: ", termination_status(model))
5   println("Solved tf = ", value(tf))
6 end
7
8 # =====
9 # Cell 7: Extract solution
10 # =====
```

```
*****
This program contains Ipopt, a library for large-scale nonlinear optimization.
Ipopt is released as open source code under the Eclipse Public License (EPL).
For more information visit https://github.com/coin-or/Ipopt
*****
```

```
75.814387 seconds (23.61 M allocations: 1.386 GiB, 0.47% gc time, 6.75% compilation time: 46% of which was recompilation)
Status: LOCALLY_INFEASIBLE
Solved tf = 30.000000292915445
```

```
([-0.00215059, 0.169106, 0.303176, 0.415674, 0.512525, 0.596556, 0.669275, 0.731505, 0.78371,
```

```
1 begin
2   opt_x = value(x); opt_y = value(y); opt_z = value(z)
3   opt_vx = value(vx); opt_vy = value(vy); opt_vz = value(vz)
4   opt_ux = value(ux); opt_uy = value(uy); opt_uz = value(uz)
5   opt_tf = value(tf)
6   ts      = supports( $\tau$ ) .* opt_tf
7
8   println("Final pos error (x,y,z): ",
9           opt_x[end]-x_L1, ", ", opt_y[end], ", ", opt_z[end])
10  println("Final vel error (vx,vy,vz): ",
11          opt_vx[end], ", ", opt_vy[end], ", ", opt_vz[end])
12  println("Maximum |z| during transfer: ", maximum(abs, opt_z))
13
14  solution = (opt_x, opt_y, opt_z, opt_vx, opt_vy, opt_vz,
15             opt_ux, opt_uy, opt_uz, ts, opt_tf)
16 end
17
18 # =====
19 # Cell 8: Trajectory plots – same as Small Creation 1
20 # =====
```

```
Final pos error (x,y,z): 0.0, -1.4327013822818394e-20, 0.0
Final vel error (vx,vy,vz): 3.9411853341594024e-20, 1.5077687306194182e-20, 0.0
Maximum |z| during transfer: 0.0
```



```

1 begin
2   p_traj = plot(opt_x, opt_y, opt_z, lw=2.5, label="Trajectory",
3               title="Earth → L1 with ONLY in-plane control (3D CR3BP)",
4               xlabel="x", ylabel="y", zlabel="z", camera=(30,30))
5   scatter!(p_traj, [-μ], [0], [0], marker=:circle, markersize=10, label="Earth")
6   scatter!(p_traj, [1-μ], [0], [0], marker=:star5, markersize=12, label="Moon")
7   scatter!(p_traj, [x_L1], [0], [0], marker=:diamond, markersize=10, color=:red,
8   label="L1")
9
10  p_xy = plot(opt_x, opt_y, lw=2, title="x-y plane",
11             aspect_ratio=:equal, legend=false)
12  scatter!(p_xy, [-μ], [0], marker=:circle, markersize=10)
13  scatter!(p_xy, [1-μ], [0], marker=:star5, markersize=12)
14  scatter!(p_xy, [x_L1], [0], marker=:diamond, markersize=10, color=:red)
15
16  p_z = plot(ts, opt_z, title="Out-of-plane deviation z(t)",
17            label="z", lw=2, color=:purple)
18  hline!(p_z, [0], linestyle=:dash, label="x-y plane")
19
20  p_ctrl = plot(ts, [opt_ux opt_uy opt_uz],
21               label=["ux" "uy" "uz (fixed=0)"],
22               title="Control inputs", ylim=(-0.11, 0.11))
23
24  display(p_traj)
25  display(p_xy)
26  display(p_z)
27  display(p_ctrl)
28
29 end
30
31 # =====
32 # Cell 9: Interval Reachability Tube
33 #   - grafted from Mini blueprint onto Small Creation 1's solution
34 #   - box sizes grow exponentially along arc length,
35 #   - reflecting the natural instability of CR3BP near L1
36 # =====

```

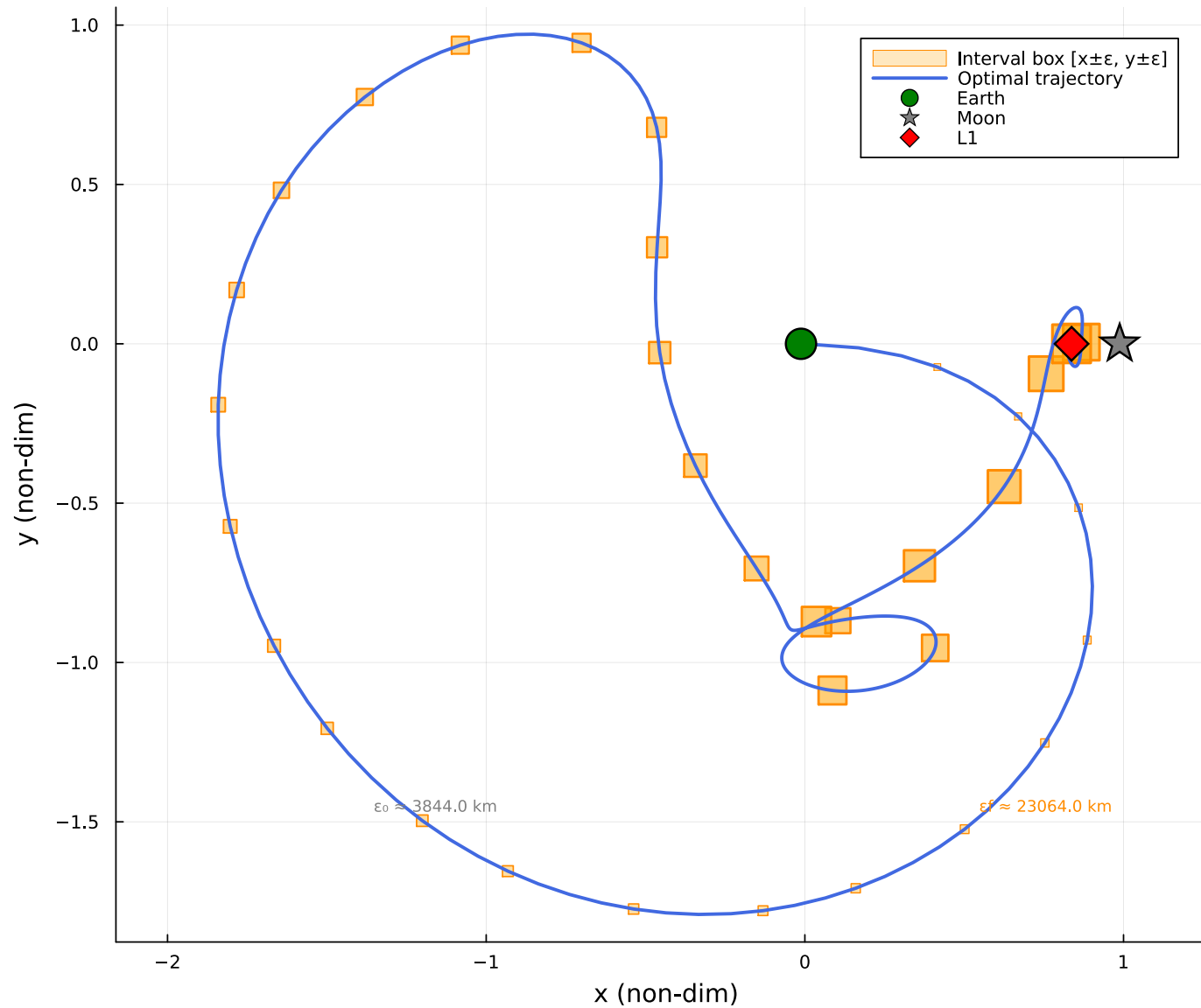
```

Plot{Plots.GRBackend() n=4}
Plot{Plots.GRBackend() n=4}

```

```
Plot{Plots.GRBackend() n=2}  
Plot{Plots.GRBackend() n=3}
```

Earth → L1: Interval Reachability Tube (x-y plane)



```

1 let
2   N = length(opt_x)
3
4   # — Cumulative arc length —————
5   arc = zeros(N)

```

```

6   for k in 2:N
7       arc[k] = arc[k-1] + sqrt((opt_x[k]-opt_x[k-1])^2 +
8                               (opt_y[k]-opt_y[k-1])^2)
9   end
10  total_arc = arc[end]
11
12  # — 35 evenly spaced sample points along arc length —
13  n_boxes = 35
14  arc_tgts = range(0, total_arc, length=n_boxes)
15  box_idx = [argmin(abs.(arc .- s)) for s in arc_tgts]
16
17  # — Box half-width: exponential growth
18  #    $\epsilon_0$  at departure,  $\epsilon_f$  at arrival (mirrors instability near L1)
19   $\epsilon_0$  = 0.010           #  $\approx 3844$  km (1% of Earth-Moon distance)
20   $\epsilon_f$  = 0.060        #  $\approx 23064$  km (6% of Earth-Moon distance)
21  box_sizes = [ $\epsilon_0 * (\epsilon_f/\epsilon_0)^{((i-1)/(n\_boxes-1))}$  for i in 1:n_boxes]
22
23  # — Build interval representation using IntervalArithmetic —
24  #   Each box centre is an Interval; width =  $2\epsilon$ 
25  interval_x = [interval(opt_x[k] - box_sizes[i], opt_x[k] + box_sizes[i])
26               for (i, k) in enumerate(box_idx)]
27  interval_y = [interval(opt_y[k] - box_sizes[i], opt_y[k] + box_sizes[i])
28               for (i, k) in enumerate(box_idx)]
29
30  println("=== Interval Reachability Boxes ===")
31  println("Boxes           : ", n_boxes)
32  println(" $\epsilon_0$  (departure) : ", round( $\epsilon_0 * 384400$ , digits=0), " km")
33  println(" $\epsilon_f$  (arrival)   : ", round( $\epsilon_f * 384400$ , digits=0), " km")
34  println("Growth factor     : ", round( $\epsilon_f/\epsilon_0$ , sigdigits=3), "x")
35  println()
36  println("Sample intervals (x, y) at first / mid / last box:")
37  for i in [1, div(n_boxes,2), n_boxes]
38      k = box_idx[i]
39      println(" i=$i  x∈", interval_x[i], "  y∈", interval_y[i])
40  end
41
42  # — Plot —
43  p = plot(title = "Earth → L1: Interval Reachability Tube (x-y plane)",
44          xlabel = "x (non-dim)",
45          ylabel = "y (non-dim)",

```

```

46         legend = :topright,
47         aspect_ratio = :equal,
48         size = (800, 700))
49
50     # Draw boxes (orange shaded rectangles, opacity grows toward L1)
51     for (i, k) in enumerate(box_idxs)
52         cx = opt_x[k]
53         cy = opt_y[k]
54         b = box_sizes[i]
55         t_n = (i - 1) / (n_boxes - 1)    # normalised position 0→1
56
57         rx = [cx-b, cx+b, cx+b, cx-b, cx-b]
58         ry = [cy-b, cy-b, cy+b, cy+b, cy-b]
59
60         plot!(p, rx, ry,
61               seriestype = :shape,
62               fillcolor = :orange,
63               fillalpha = 0.25 + 0.35*t_n,
64               linecolor = :darkorange,
65               linewidth = 0.6 + 1.2*t_n,
66               label      = i == 1 ? "Interval box [ $x \pm \epsilon$ ,  $y \pm \epsilon$ ]" : "")
67     end
68
69     # Optimal trajectory on top
70     plot!(p, opt_x, opt_y,
71           lw      = 2.2,
72           color = :royalblue,
73           label = "Optimal trajectory")
74
75     # Celestial bodies
76     scatter!(p, [-μ], [0], marker=:circle, markersize=11,
77              color=:green, label="Earth")
78     scatter!(p, [1-μ], [0], marker=:star5, markersize=13,
79              color=:gray, label="Moon")
80     scatter!(p, [x_L1], [0], marker=:diamond, markersize=11,
81              color=:red, label="L1")
82
83     # Annotations
84     annotate!(p, -1.35, -1.45,
85              text(" $\epsilon_0 \approx \$(round(\epsilon_0 * 384400, digits=0)) \text{ km}$ ", 7, :left, :gray))

```

```
86     annotate!(p, 0.55, -1.45,  
87               text("εf ≈ $(round(εf*384400, digits=0)) km", 7, :left, :darkorange))  
88  
89     display(p)  
90     p
```

```
=== Interval Reachability Boxes ===
```

```
Boxes           : 35  
ε₀ (departure)  : 3844.0 km  
εf (arrival)    : 23064.0 km  
Growth factor   : 6.0×
```

```
Sample intervals (x, y) at first / mid / last box:
```

```
 i=1  x∈[-0.0121506, 0.00784941]_com  y∈[-0.01, 0.01]_com  
 i=17 x∈[-1.8063, -1.75983]_com  y∈[0.145669, 0.192144]_com  
 i=35 x∈[0.776915, 0.896915]_com  y∈[-0.06, 0.06]_com
```

```
Plot{Plots.GRBackend() n=39}
```